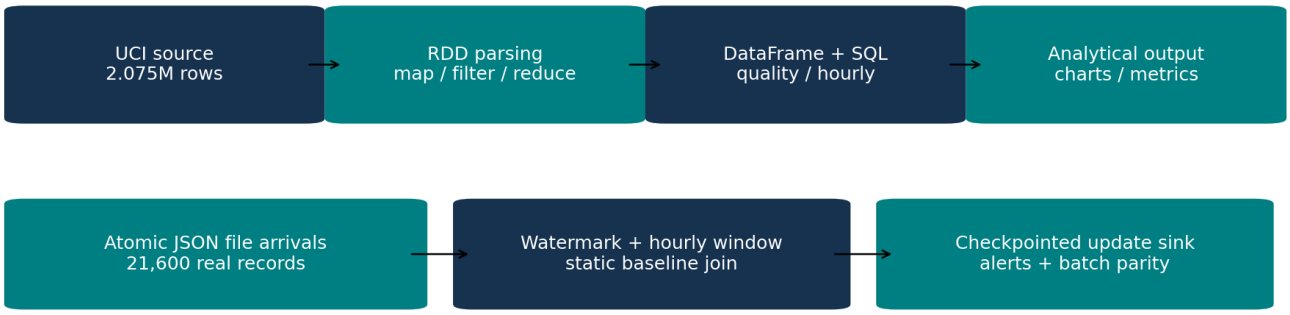
# GridPulse

## Event-Time Reliability and Seasonality-Aware Anomaly Detection for Household Energy Telemetry with Apache Spark

### Mining of Massive Datasets | Assignment 2 | CAL 2 technical report

Execution-backed prototype | UCI Household Power Consumption | PySpark 3.5.3




**Measured scope:** 2,075,259 raw readings; 2,049,280 usable readings; 21,600 real readings replayed through Structured Streaming.

**Administrative requirement:** confirm CAL 1 dataset continuity or instructor approval for this choice, and merge these sections into the actual CAL 1 report. Group metadata is editable in submission.json. No approval or personal details have been assumed.

**Research status:** a reproducible systems and analytics prototype. Conference readiness, algorithmic novelty and natural fault accuracy have not been established.

# 1. Abstract and motivation

GridPulse implements an end-to-end Spark pipeline over 2,075,259 minute-level household electricity measurements. It ingests semicolon-delimited source records through RDDs, applies explicit parsing and quality checks, converts accepted records into a typed DataFrame, and produces energy summaries through Spark SQL. The implementation independently reconciles RDD and SQL monthly totals. A file-arrival simulation replays 21,600 real readings through event-time hourly windows, scores them against a static seasonality-aware baseline, and validates outputs after a checkpoint restart.

The analytical experiment compares global, hour-of-day and hour-of-week robust baselines. Models use 2006-2008 data; thresholds are calibrated using 2009; 2010 remains held out. Controlled positive spikes and six-hour shifts evaluate sensitivity at three amplitudes and three seeds. A separate four-event probe demonstrates the retention effect of three watermark budgets. These experiments connect analytic quality to streaming semantics without assuming natural anomaly labels exist.

**Problem:** a high power reading may represent expected evening usage or an unusual departure from a temporal pattern. Furthermore, a delayed record may be omitted after state eviction. Therefore a useful analytics pipeline must document both its anomaly calibration and the completeness of its streaming aggregates.

**Research question:** How does seasonal conditioning change sensitivity to controlled energy perturbations, and how does event-time lateness affect the evidence retained by a streaming pipeline? A real-data delayed replay also measures aggregate error and alert changes; detection delay under sustained load remains future work.

Related work includes transformer-based energy anomaly detection with synthesized data [3], critiques of anomaly benchmark validity [4], learned anomaly generation [5], and comparisons questioning whether complex models improve on conventional baselines [6]. This prototype uses simple, inspectable baselines to focus on event-time evidence retention. It does not claim to outperform those papers or introduce a new detector.

**Objectives:** cover the Spark architecture, RDD transformations and actions, typed DataFrames, cleaning, grouping and SQL, processing workflow, and Structured Streaming rubric; preserve measured results and reproducibility evidence; identify the next experiments needed for credible research.

# 2. Dataset selection and provenance

| Candidate | Fit and tradeoff |
|---|---|
| UCI Household Power Consumption - selected | 2.075M timestamps, real missing readings, seven electrical measures; modest download, no authentication, reproducible replay; one household and no fault labels. |
| NYC TLC taxi records - considered | Large timestamped transaction data; strong spatial analytics, but trip completion versus event arrival semantics and external context require additional design. Not downloaded. |
| Network intrusion benchmarks - considered | Rich security labels but often suffer capture-specific leakage and unclear realistic stream timing. Not downloaded. |

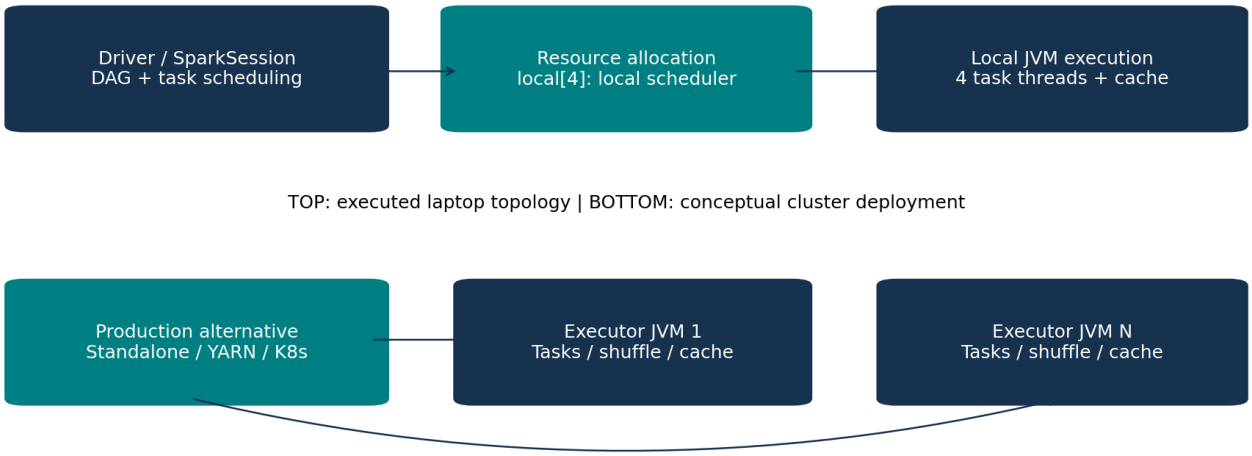
Selected source: Georges Hebrail and Alice Berard, *Individual Household Electric Power Consumption*, UCI Machine Learning Repository, DOI 10.24432/C58K54. Licence: Creative Commons Attribution 4.0. The primary source describes one home in Sceaux, France, sampled every minute from December 2006 through November 2010 [1].

Input columns: Date, Time, global active power (kW), reactive power (kVAR), voltage (V), current (A), and kitchen/laundry/heating-cooling submeters (Wh per minute). One-minute active energy is active_kw / 60 kWh. Other-appliance energy is active_kw * 1000 / 60 minus the three submeters in Wh. Negative residuals are counted and retained. Their physical cause is unknown; rounding or measurement alignment would require supporting metadata.

The source archive is downloaded by scripts/download.py. Downloads use retries and atomic rename. Cached ZIP/text bytes must match pinned SHA-256 values, and ZIP CRC is checked. A changed extracted file is restored from the verified archive; an incomplete archive is downloaded again. Validated hashes are written to data/raw/manifest.json. Original source data is retained. Missing, malformed, non-finite or negative measurement records are rejected; duplicates are removed by timestamp. Rejected rows are accounted for rather than silently changing the denominator.

Streaming means replaying a historical timestamped source, as explicitly permitted by the assignment. It is not a live utility feed. Source times have no timezone offset: Spark uses a UTC session to preserve a deterministic wall-clock interpretation. This does not establish true UTC event instants; French daylight-saving transitions cannot be reconstructed from missing timezone metadata.

# 3. Spark environment and architecture (CO2)




TOP: executed laptop topology | BOTTOM: conceptual cluster deployment

Executed stack: Python 3.11.15, Apache Spark 3.5.3, Java 17, local[4]. Host: macOS-27.2-arm64-arm-64bit. Hardware details are recorded in outputs/environment.json. Dependencies are isolated in .venv and captured in requirements.lock.txt. JAVA_HOME is configured by scripts/env.sh; no shell profile changes are required.

The driver constructs SparkSession, holds the SparkContext, builds the logical plan or RDD lineage, schedules stages and coordinates results. In a distributed deployment a cluster manager such as Standalone, YARN or Kubernetes allocates executor resources. Executors run partition tasks, cache data and exchange shuffle outputs. The executed local[4] deployment has four task threads in one local JVM; it is not four machines or a multi-executor cluster.

Spark transformations are lazy. filter and map build a narrow-dependency lineage; reduceByKey and grouping introduce shuffle dependencies, splitting stage boundaries. Actions such as count, collect and write trigger jobs, with one task per partition in a stage. DataFrame logical plans are optimized by Catalyst and may be revised by adaptive query execution. Python RDD parsing incurs Python worker and serialization overhead that built-in DataFrame expressions can avoid.

The batch source uses at least eight partitions and the SQL shuffle target is eight. Intermediate parsed RDDs and clean DataFrames are cached to avoid repeated work. Only small aggregated results and bounded hourly/test/replay data are collected to the driver. The full clean dataset remains distributed in Parquet. Event logs and query plans are retained for execution-model inspection; a running application exposes a transient Spark UI on localhost:4040.

# 4. RDD ingestion and validation (CO2)

Implementation: src/batch.py. SparkContext.textFile loads the actual UCI source as an RDD. filter removes the header. map(parse_line) validates nine fields, parses the timestamp and converts seven finite nonnegative measurements. filter removes failed parses. This gives a direct and auditable RDD entry point rather than creating an RDD indirectly from a DataFrame.

flatMap emits a (submeter, 1) pair for every positive kitchen, laundry or HVAC measurement. A single record may emit zero to three keys. reduceByKey sums active minutes and demonstrates map-side combining. A second map and reduceByKey compute monthly (energy sum, valid-minute count) tuples. sortByKey orders the output. count and collect are the explicit actions used for validation and small aggregate reporting.

| Check | Measured value |
|---|---|
| source_rows | 2075259 |
| valid_rows | 2049280 |
| rejected_rows | 25979 |
| duplicate_timestamps | 0 |
| negative_unmetered_minutes | 988 |
| rdd_sql_reconciliation | PASS |
| **Submeter** | **Minutes with positive measurement** |
| laundry | 612450 |
| kitchen | 169105 |
| hvac | 1197188 |

The RDD monthly energy and valid-minute results are compared with independent SQL results for every month. Month key sets and cardinality must agree before values are compared. Counts must agree exactly and energy must agree within 0.001 kWh after the SQL display rounding. This is a meaningful cross-API correctness check, not a comparison of two copies of the same output. Regression tests cover parsing, timezone stability, interrupted downloads, source tampering, nonfinite streaming input, scoring eligibility, missing profiles and key-set mismatches.

Evidence: outputs/rdd_results.json contains source counts, reductions and RDD lineage; outputs/logs/tests.log records parser tests; outputs/quality.json records reconciliation. Source rows with absent measurements remain represented in the raw file and in rejected-row accounting.

# 4b. Data-quality diagnostics

| Parsing result | Rows |
|---|---|
| accepted | 2049280 |
| missing_measurement | 25979 |

Rejections.json contains raw examples and explicit reason categories. Monthly coverage divides accepted observations by raw source timestamps in that month, retaining partial endpoint months. Coverage is not a prediction of missing consumption. Missing_runs.csv lists contiguous rejected-reading intervals, so a file containing 3,600 valid observations can span much longer than 60 clock hours.

| Month | Source minutes | Valid minutes | Coverage |
|---|---|---|---|
| 2010-08 | 44640 | 37414 | 83.81% |
| 2010-09 | 43200 | 37963 | 87.88% |
| 2007-04 | 43200 | 39477 | 91.38% |
| 2009-06 | 43200 | 39895 | 92.35% |
| 2010-01 | 44640 | 41509 | 92.99% |
| 2010-03 | 44640 | 42613 | 95.46% |

## Longest missing-measurement runs

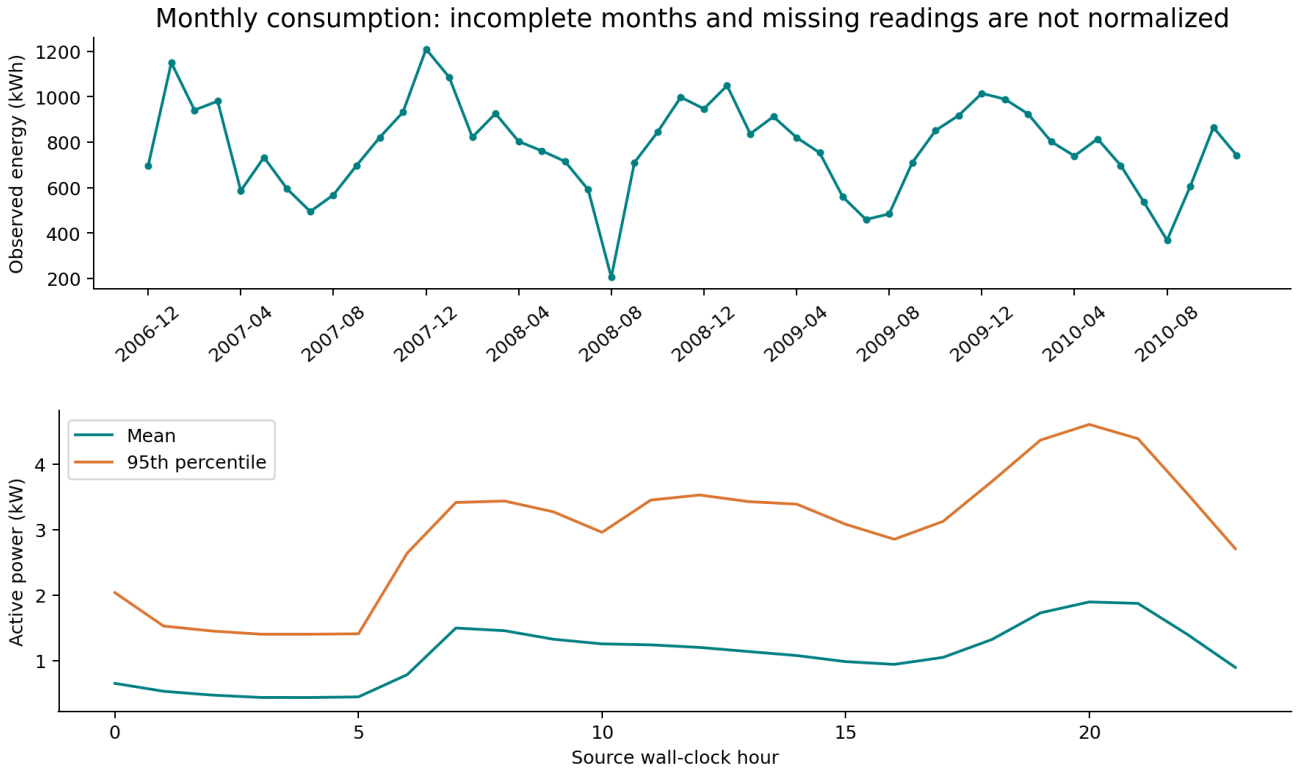
| Start | End | Minutes |
|---|---|---|
| 2010-08-17 21:02:00 | 2010-08-22 21:27:00 | 7226 |
| 2010-09-25 03:56:00 | 2010-09-28 19:12:00 | 5237 |
| 2007-04-28 00:21:00 | 2007-04-30 14:23:00 | 3723 |
| 2009-06-13 00:30:00 | 2009-06-15 07:34:00 | 3305 |
| 2010-01-12 14:53:00 | 2010-01-14 19:01:00 | 3129 |

The residual submeter calculation is below -0.1 Wh for 988 minutes. Its measured minimum is -2.40 Wh. Residual_diagnostics.json records quantiles. These flags remain available for inspection without assuming a physical cause or silently clamping them to zero.

# 5. DataFrames, Spark SQL and dataflow (CO4)

Accepted RDD tuples are converted using an explicit StructType: timestamp followed by seven doubles. Cleaning removes timestamp duplicates and adds one-minute kWh and residual submeter energy. Grouping into hourly features produces valid-minute coverage, mean kW, observed kWh, hour-of-day and hour-of-week. Detection uses hours with at least 54 valid minutes; no future interpolation is performed. Monthly totals are observed totals and must not be interpreted as complete consumption when coverage is incomplete.

The saved SQL queries implement monthly trends, top days with at least 1,380 observed minutes, hourly usage profiles with approximate 95th percentiles, and submeter energy allocation. They demonstrate filtering (WHERE/HAVING), aggregation, GROUP BY, ordering and a temporary SQL view. Query text, result CSVs and the optimized/physical monthly query plan are retained.




**Observed patterns:** the largest observed monthly total is 1,210.1 kWh in 2007-12. Mean active power peaks at source hour 20:00 (1.90 kW). This is descriptive evidence from one household; different coverage and month lengths prevent interpreting raw monthly ranks as weather-adjusted or population-level trends.

Dataflow: immutable raw archive -> RDD parsing/reductions -> typed clean DataFrame -> Parquet hourly features and SQL outputs -> calibrated baselines -> replayed event-time windows -> saved scores, validation metrics and figures. Architecture and workflow diagrams are also exported as separate PNG figures.

# 6. Structured Streaming implementation (CO4)

Implementation: src/streaming.py. A producer atomically renames JSON files into a watched directory. Six files each contain 3,600 real readings. The consumer uses readStream with an explicit timestamp/double schema, maxFilesPerTrigger=1, invalid-row filtering, a two-hour event-time watermark and one-hour tumbling windows. It rejects null, nonfinite and negative power, computes record count, observed kWh and mean kW, then joins a static hour-of-week profile. The model choice is a predetermined demonstrator, not a selection based on test labels. Calibration thresholds live in a separate calibration.json artifact derived only from 2009.

A foreachBatch update sink writes each batch to a stable batch-ID directory in overwrite mode. This is idempotent for local single-writer retries of the same batch, but does not prove arbitrary external-sink exactly-once delivery. Latest snapshots are selected by window start and maximum batch ID, so update rows are not summed across revisions. The exported view labels a window finalized when its end is at or before the last observed query watermark, and provisional otherwise. Scoring status is separate: eligible, insufficient_coverage or missing_profile. Windows below 54 valid readings have a null score and alert, rather than a false prediction.

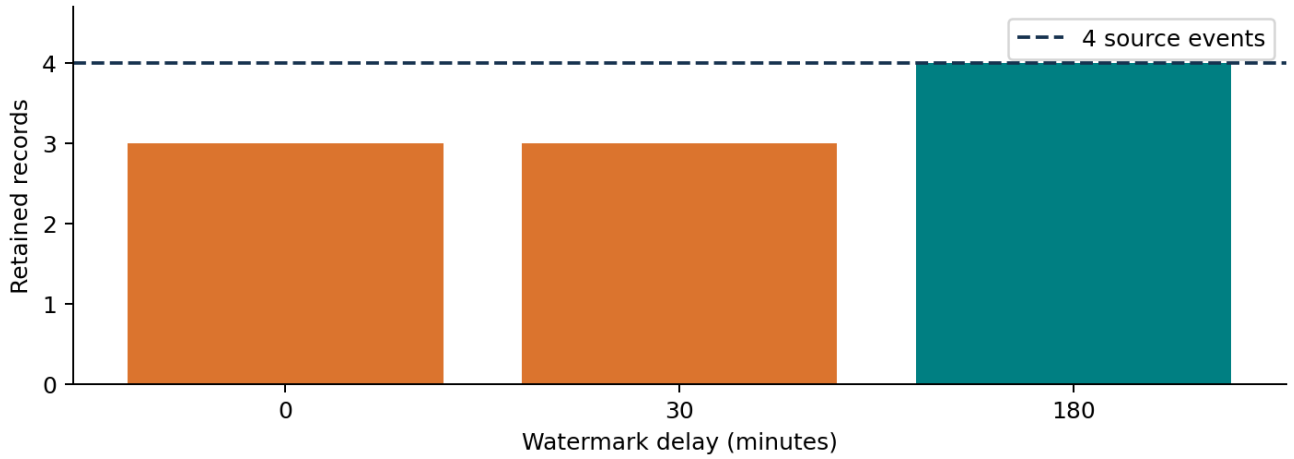
The driver waits for each micro-batch to drain before publishing the next file. This is an accelerated simulation of periodic file arrivals, not a processing-rate or end-to-end latency benchmark. A one-second processing trigger is configured. The query stops after file three, restarts with the same checkpoint and processes files four through six. The final latest-window view is compared to an independent pandas grouping of the same real input readings.

| Replay validation | Measured result |
|---|---|
| real_records_replayed | 21600 |
| hourly_windows | 361 |
| max_energy_error_kwh | 2.220446049250313e-15 |
| checkpoint_restart_after_files | 3 |
| batch_stream_parity | PASS |
| eligible_windows | 359 |
| insufficient_coverage_windows | 2 |

Checkpoint directories preserve source offsets and aggregation state. They must be paired with the same source and compatible query topology. This scripted experiment clears only its own outputs/streaming directory at the start for repeatability, then tests recovery within that run. It does not test a machine failure or transactional sink failure.

Progress JSON retains input rows, event-time watermark, batch durations, state memory, state-row counts and dropped-late-row counters. This supplies direct evidence of incoming records, aggregation and watermark behavior. Actual progress JSON, selected completed Spark event logs, stage metrics and genuine browser screenshots accompany the report. Tables and charts remain separate from screenshots. The live-demo script keeps the UI open without resetting submission evidence.

# 7. Late-data experiment and interpretation




| watermark_minutes | input_records | accepted_records | dropped_by_watermark |
|---|---|---|---|
| 0 | 4 | 3 | 1 |
| 30 | 4 | 3 | 1 |
| 180 | 4 | 4 | 0 |

The controlled probe has arrivals at 00:00, 00:50, 02:00, followed by a delayed 00:10 record, all on the same date. Every event has a 1 kW reading. The first two events share a one-hour window. Advancing the maximum event time to 02:00 allows short-watermark queries to evict the first window before the delayed event arrives. A 180-minute budget retains that window and accepts the delayed event. The late probe is synthetic; the principal 21,600-record replay is real historical input.

A watermark is computed from maximum observed event time minus the delay. State eviction and row-dropping rules depend on the window boundary and the watermark available from previous micro-batches. Watermark time does not advance just because the wall clock advances. Records within the lateness budget are protected by the documented semantics; records beyond it may or may not be accepted depending on operator and state progress.

These results establish event-time retention semantics for this deterministic sequence. They do not establish an optimal production watermark. A larger budget can preserve more delayed evidence but increases retained state and can delay final append-mode output. The update-mode snapshots used here are provisional; finalization latency is not measured. A publication experiment should sample realistic arrival delays and compare alert delay and memory under fixed input rates.

# 7b. Real-data delayed replay: reliability versus state

| Budget (h) | Source records | Retained records | Retention | Energy error (kWh) | Alert changes | Peak state (bytes) |
|---|---|---|---|---|---|---|
| 2 | 21600 | 19514 | 0.90343 | 45.60630 | 0 | 19648 |
| 72 | 21600 | 21366 | 0.98917 | 4.62953 | 0 | 45152 |
| 168 | 21600 | 21600 | 1.00000 | 0.00000 | 0 | 73184 |

This second experiment changes arrival order while keeping every source timestamp and measurement intact. Every tenth record is withheld from each 3,600-record file and released with the next file. Each file contains 60 hours of valid observations, but spans more clock time across missing-data gaps; the final deferred set is released as a seventh file. All three watermark configurations receive the same 21,600 real readings. No synthetic late timestamps or finalizer records are inserted.

A two-hour budget can evict hourly state before many delayed measurements arrive. A 72-hour budget can still lose evidence because missing-data gaps lengthen file spans. A 168-hour budget protects this one-file delay schedule and is asserted to preserve every record and reproduce offline energy within numerical tolerance. Metrics compare the latest streaming window snapshots against the chronological reference: retained record fraction, sum of absolute hourly energy errors, alert disagreements, and peak state memory observed in progress logs.

The energy-error metric is an hourly absolute-error total, not relative error and not household billing loss. Alert disagreements compare only windows eligible in both runs; unscored windows are reported separately. All three unperturbed runs have zero alert disagreements. Thus this experiment demonstrates retention and energy fidelity, without establishing a fault-detection gain. Peak state memory measures the streaming state operator rather than total JVM resident memory.

This demonstrates a concrete retention/aggregate-fidelity/state tradeoff on real data. It does not establish that 168 hours is optimal or that larger watermarks always improve performance. Arrival delay is deterministic, source time is accelerated, and only three budgets are tested at this scale. The next section tests controlled minute-level faults. Random delay distributions and detection latency under sustained load remain future work.

# 7c. Controlled faults under delayed arrivals

A separate experiment inserts a +10 kW synthetic burst into every tenth minute of 12 randomly selected eligible hours (seed 2026). All other source values remain unchanged. The same arrival schedule delays every tenth record by one file, so late evidence overlaps the perturbations. The amplitude is a stress-test parameter, not a calibrated model of household faults.

An independent pandas reference aggregates the perturbed minute records and applies the same predetermined profile and 2009 threshold. The stream uses 2-hour and 168-hour budgets. Labels identify hours containing an injected perturbation. Non-injected hours have no verified natural fault labels, so confusion counts measure agreement with the controlled injection task only.

| Budget (h) | TP | FP | FN | Recall | F1 | Changed alerts |
|---|---|---|---|---|---|---|
| offline | 1 | 2 | 11 | 0.083 | 0.133 | 0 |
| 2 | 0 | 2 | 12 | 0.000 | 0.000 | 1 |
| 168 | 1 | 2 | 11 | 0.083 | 0.133 | 0 |

Alert changes compare windows eligible in both the offline and streaming views. The end-to-end recall denominator includes injected hours that become unscored, and reports those unscored fault hours separately in the CSV. The 168-hour replay is asserted to reproduce offline mean power and eligible alert decisions. This protocol tests final snapshot quality; a stopped-query comparison cannot measure first-alert latency.

**Measured result:** the offline reference detects 1 of 12 injected hours (recall 8.3%). The two-hour replay detects 0 and changes 1 eligible decisions. The seven-day replay matches the offline reference. This shows that delayed evidence can change an alert in this controlled case, while the low offline recall exposes the limits of hourly averaging and this static detector.

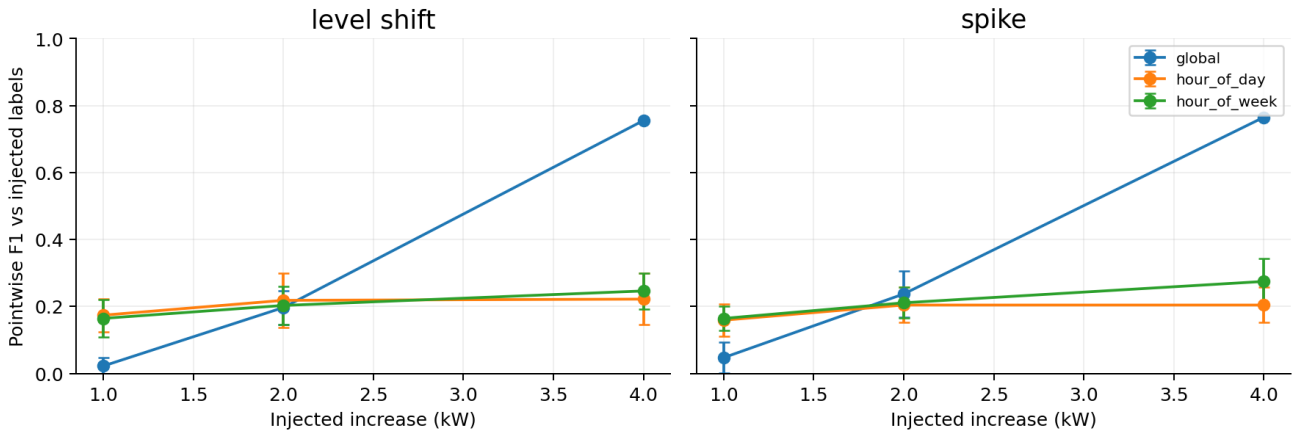
Hourly averaging can dilute short faults. A negative result or missed event therefore motivates future peak/residual features rather than supporting a claim of reliable fault detection. The saved protocol lists every injected hour and the number of perturbed minute records, allowing the comparison to be independently reconstructed.

# 8. Seasonality-aware anomaly experiment

Let x be mean hourly kW. Each model estimates a training median m and median absolute deviation MAD, with scale s = max(1.4826 * MAD, 0.05 kW). The score is |x - m| / s. The global model uses one profile; hour-of-day uses 24 profiles; hour-of-week uses 168 profiles. Approximate medians use Spark percentile_approx with accuracy 10,000. Each threshold is the approximate 99th percentile of that model's 2009 calibration scores, never optimized on injected 2010 labels.

| Split | Hours with >=54 valid minutes |
|---|---|
| Training: through 2008 | 17837 |
| Calibration: 2009 | 8686 |
| Held-out test: 2010 | 7620 |

Controlled scenarios inject +1, +2 or +4 kW into approximately 1% of test hours. Point spikes use independent sampled hours; shifts use sampled non-overlapping six-row blocks. Seeds 7, 19 and 41 generate identical perturbations across all models. Shift candidates must span six consecutive clock hours, and blocks are non-overlapping, so missing-hour gaps cannot become false continuous events. The perturbation amplitude is absolute and model-independent to avoid favoring a seasonal baseline by construction.




Precision, recall, F1 and background alert rate are pointwise against injection indicators. Non-injected readings are not confirmed normal: a flag on them counts as a false positive only for this injection task. No point-adjusted scoring is used. Error bars show sample standard deviation over three seeds; they are not confidence intervals. Saved outputs retain all confusion counts and thresholds.

# 9. Results, scalability and limitations

| Model | Precision | Recall | F1 | Background alert rate |
|---|---|---|---|---|
| global | 0.2978 | 0.4096 | 0.3372 | 0.0062 |
| hour_of_day | 0.1715 | 0.2316 | 0.1968 | 0.0108 |
| hour_of_week | 0.17 | 0.2769 | 0.2105 | 0.013 |

**Measured comparison:** global has the highest average F1 in this controlled suite (0.3372). This pooled average weights each amplitude and fault type equally. Seasonal models do better on small injections, while the global model is stronger on large injections. Calibration controls the 2009 score distribution, not the 2010 background alert rate; household behavior drift and profile granularity are plausible explanations that require additional tests, not established causes.

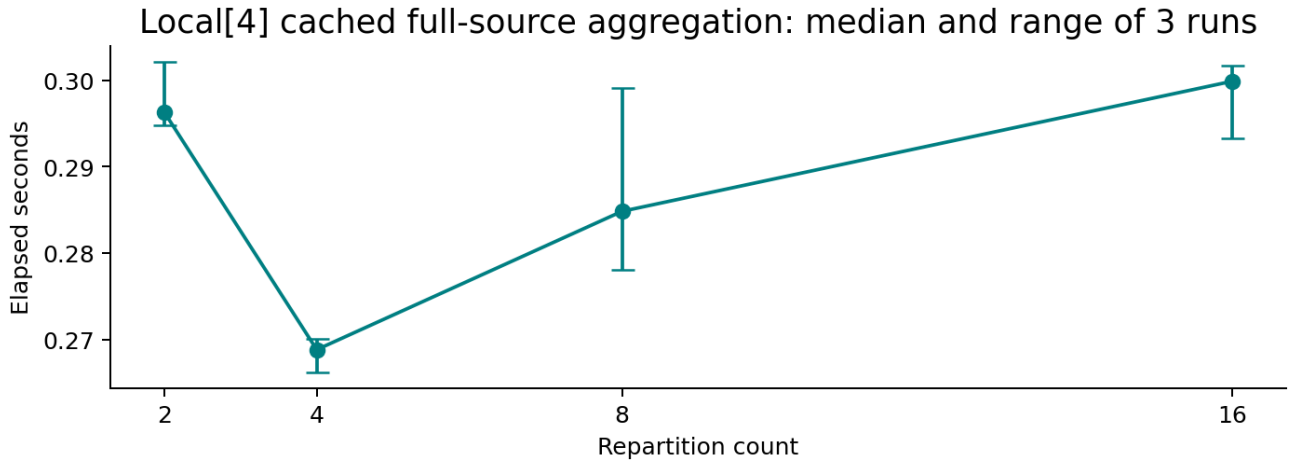
These averages combine both perturbation types, all amplitudes and three seeds. They summarize this controlled suite only; the per-scenario CSV and plot are the appropriate basis for examining strengths and weaknesses. A more granular seasonal profile is not guaranteed to improve detection: sparse profile estimates and changing household behavior can increase background alerts. Results should be reported even when the simpler global baseline is stronger.

## Sensitivity depends on injection strength

| Model | F1: +1 kW spike | F1: +4 kW spike |
|---|---|---|
| global | 0.047 | 0.765 |
| hour_of_day | 0.159 | 0.204 |
| hour_of_week | 0.164 | 0.275 |

Three seeds provide paired repeatability, not confidence intervals. A deployment-weighted average would need a justified fault distribution. Changing model selection after seeing 2010 results would consume the held-out test set, so the streaming demonstrator retains its preselected weekly profile.

# 9b. Execution and partition sensitivity




The partition experiment operates on all 2,049,280 clean minute records, with narrow columns cached, then repartitions and aggregates mean power by hour-of-week. It uses counts 2, 4, 8 and 16, performs one warm-up per count outside the timing, randomizes 12 measured trials, and reports median and range of three timings. Driver collection is bounded to 168 aggregate rows. These are local partition-sensitivity timings including shuffle and scheduling, not distributed speedup or strong-scaling measurements. OS caches, JIT, AQE and shared laptop load remain confounders.

## Local data-volume experiment

| Input rows | Median (s) | Min (s) | Max (s) |
|---|---|---|---|
| 512284 | 0.099 | 0.095 | 0.101 |
| 1024255 | 0.158 | 0.158 | 0.160 |
| 2049280 | 0.287 | 0.277 | 0.347 |

The volume experiment fixes local[4] and eight repartitioned inputs, samples 25%, 50% and 100% of valid source rows with seed 2026, and warms each cached sample before three timed aggregations. It shows local workload sensitivity. Increasing data volume while fixing one JVM does not measure multi-executor scaling.

Limitations: one household, historical data, unknown true fault labels, missing readings and incomplete months, unresolved daylight-saving metadata, static profiles subject to drift, small injected-anomaly family, three seeds, no distributed cluster, no realistic backpressure workload and only one deterministic lateness schedule. The new joint fault replay measures final snapshot decisions, not alert latency. Full-source batch processing and replay exceed the assignment size minimum, but that alone does not establish research novelty.

# 10. Rubric mapping, reproduction and conclusion

| Rubric / marks | CO | Implementation and evidence |
|---|---|---|
| Architecture and workflow / 3 | CO2 | Section 3; architecture.png, workflow.png, event_logs, query_plan.txt |
| RDD implementation / 4 | CO2 | src/batch.py; map/filter/flatMap/reduceByKey; rdd_results.json; reconciliation and parser tests |
| DataFrames and SQL / 3 | CO4 | Explicit schema, cleaning, hourly grouping, saved SQL and result CSVs; Section 5 |
| Structured Streaming / 2 | CO4 | src/streaming.py; real replay, window aggregates, checkpoint restart, progress logs, watermark probe |
| Results and discussion / 3 | CO4 | Measured quality, temporal experiment, late-data evidence, partition timings, limitations and research plan |

**Reproduce:** source scripts/env.sh; bash scripts/run_all.sh. The script validates/downloads the source, runs tests, processes the full dataset, executes the experiment suite and streaming replay, then builds this PDF. Raw data, environment and result directories are excluded from source control. requirements.lock.txt captures the executed Python stack. On a new machine install Java 17 and use scripts/setup.sh first.

**Evidence guide:** outputs/logs for execution; outputs/sql for queries and results; outputs/streaming for file arrivals, snapshots, checkpoints and progress; outputs/figures for report graphics. docs/VIVA_AND_DEMO.txt contains a demonstration sequence and technical answers. docs/RESEARCH_PLAN.txt defines the publication gate and missing CAL 1 continuity.

**Conclusion:** GridPulse demonstrates the complete Spark workflow with tested cross-API consistency, a real-data streaming replay and explicit late-arrival semantics. Its research value is currently the reproducible experimental foundation. A credible conference submission requires stronger labelled datasets, competing methods, realistic delayed arrivals, paired uncertainty estimates and actual multi-executor measurements. The present report makes no guarantee of publication or marks.

# 11. References and submission checklist

[1] G. Hebrail and A. Berard. Individual Household Electric Power Consumption. UCI Machine Learning Repository. DOI: 10.24432/C58K54. https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption. CC BY 4.0. Primary metadata verified during this build.

[2] Apache Spark 3.5.3 documentation: RDD Programming Guide; Spark SQL, DataFrames and Datasets Guide; Structured Streaming Programming Guide. https://spark.apache.org/docs/3.5.3/ . Implementation references; consult the versioned documentation for operator guarantees.

[3] T. Mu, Y. Yu, G. Feng, H. Luo and H. Yang. Detecting anomalous electricity consumption with transformer and synthesized anomalies. PeerJ Computer Science (2023). DOI: 10.7717/peerj-cs.1721. https://pmc.ncbi.nlm.nih.gov/articles/PMC10702936/ . Related domain work; its reported dataset and scores are not directly comparable to this project.

[4] Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress. arXiv:2009.13807. https://arxiv.org/abs/2009.13807 . Motivates explicit benchmark limitations and honest evaluation.

[5] GenIAS: Generator for Instantiating Anomalies in Time Series. arXiv:2502.08262. https://arxiv.org/abs/2502.08262 . Related synthetic anomaly generation; GridPulse uses simple controlled injections, not GenIAS.

[6] Do Deep Neural Networks Contribute to Multivariate Time Series Anomaly Detection? arXiv:2204.01637. https://arxiv.org/abs/2204.01637 . Context for retaining conventional baselines rather than assuming complexity improves results.

**Before submission:** confirm the CAL 1 dataset/report match; add actual group details and contributions; review the supplied genuine Spark and result-view screenshots and rehearse the live demonstration; review the report and be able to explain the saved negative results. Exported result figures are actual measurements but are not claimed as UI screenshots.

# 12. Query and streaming result evidence

The following figures render actual saved result CSVs. They are execution evidence, not browser or terminal screen captures. Genuine browser screenshots of the Spark UI and the result evidence document follow in the screenshot appendix.

## Monthly Spark SQL output (first twelve rows)

| month | valid_minutes | observed_kwh | mean_kw |
|---|---|---|---|
| 2006-12 | 21992 | 696.888 | 1.9013 |
| 2007-01 | 44638 | 1150.1977 | 1.546 |
| 2007-02 | 40318 | 941.4814 | 1.4011 |
| 2007-03 | 44639 | 981.0365 | 1.3186 |
| 2007-04 | 39477 | 586.3578 | 0.8912 |
| 2007-05 | 44640 | 733.4812 | 0.9859 |
| 2007-06 | 43157 | 594.7138 | 0.8268 |
| 2007-07 | 44509 | 495.0638 | 0.6674 |
| 2007-08 | 44618 | 568.2743 | 0.7642 |
| 2007-09 | 43198 | 697.8768 | 0.9693 |
| 2007-10 | 44638 | 821.2728 | 1.1039 |
| 2007-11 | 43198 | 931.9774 | 1.2945 |

## Watermarked streaming aggregation results

| watermark_minutes | input_records | accepted_records | dropped_by_watermark |
|---|---|---|---|
| 0 | 4 | 3 | 1 |
| 30 | 4 | 3 | 1 |
| 180 | 4 | 4 | 0 |

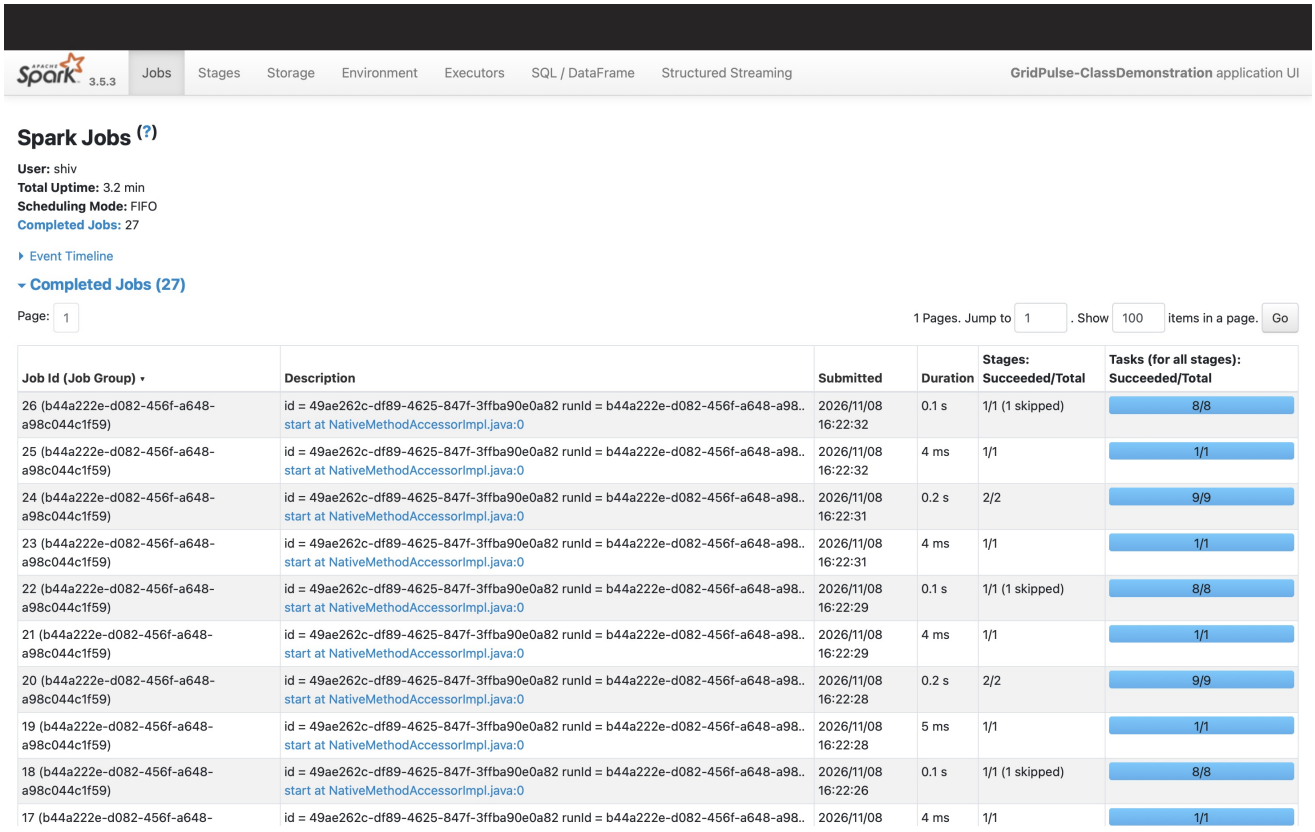
## Example saved query

```
SELECT date_format(event_time,'yyyy-MM') month, COUNT(*) valid_minutes,
        ROUND(SUM(energy_kwh),4) observed_kwh, ROUND(AVG(active_kw),4) mean_kw
        FROM energy GROUP BY 1 ORDER BY 1
```

# 13. Genuine browser evidence

Screenshots capture the running Spark application and saved query/stream results displayed in a browser. The latter show actual output CSVs and progress JSON, and are labelled as recorded evidence. The source data, query files and execution logs accompany them.

## Running Spark Jobs UI



Spark 3.5.3 | Jobs | Stages | Storage | Environment | Executors | SQL / DataFrame | Structured Streaming | GridPulse-ClassDemonstration application UI

**Spark Jobs** (?)

**User:** shiv
**Total Uptime:** 3.2 min
**Scheduling Mode:** FIFO
**Completed Jobs:** 27

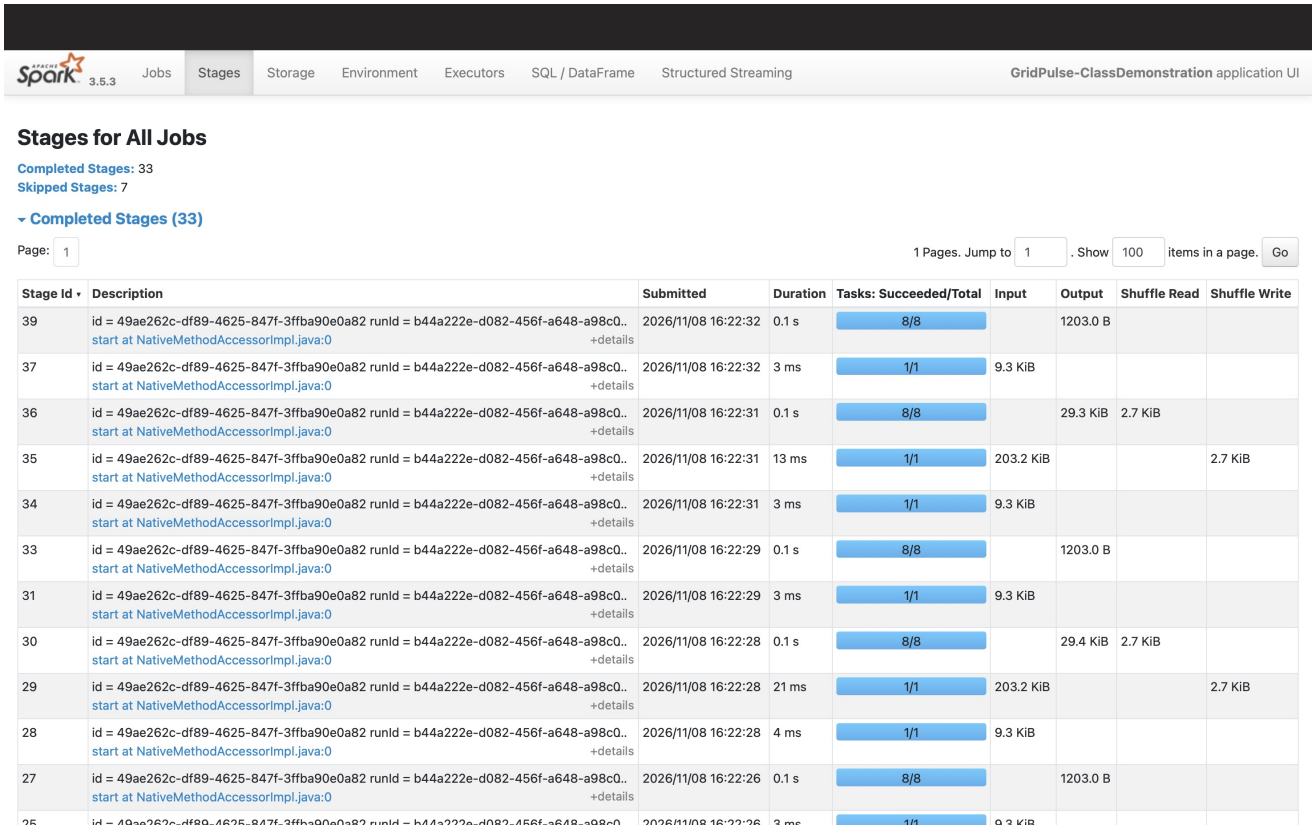
▸ Event Timeline

▾ **Completed Jobs (27)**

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

| Job Id (Job Group) | Description | Submitted | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total |
|---|---|---|---|---|---|
| 26 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:32 | 0.1 s | 1/1 (1 skipped) | 8/8 |
| 25 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:32 | 4 ms | 1/1 | 1/1 |
| 24 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:31 | 0.2 s | 2/2 | 9/9 |
| 23 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:31 | 4 ms | 1/1 | 1/1 |
| 22 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:29 | 0.1 s | 1/1 (1 skipped) | 8/8 |
| 21 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:29 | 4 ms | 1/1 | 1/1 |
| 20 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:28 | 0.2 s | 2/2 | 9/9 |
| 19 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:28 | 5 ms | 1/1 | 1/1 |
| 18 (b44a222e-d082-456f-a648-a98c044c1f59) | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. start at NativeMethodAccessorImpl.java:0 | 2026/11/08 16:22:26 | 0.1 s | 1/1 (1 skipped) | 8/8 |
| 17 (b44a222e-d082-456f-a648- | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98.. | 2026/11/08 | 4 ms | 1/1 | 1/1 |

# Spark Stages UI with task and shuffle evidence



Spark 3.5.3 | Jobs | Stages | Storage | Environment | Executors | SQL / DataFrame | Structured Streaming | GridPulse-ClassDemonstration application UI

## Stages for All Jobs

**Completed Stages:** 33
**Skipped Stages:** 7

**Completed Stages (33)**

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

| Stage Id | Description | Submitted | Duration | Tasks: Succeeded/Total | Input | Output | Shuffle Read | Shuffle Write |
|---|---|---|---|---|---|---|---|---|
| 39 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:32 | 0.1 s | 8/8 | | 1203.0 B | | |
| 37 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:32 | 3 ms | 1/1 | 9.3 KiB | | | |
| 36 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:31 | 0.1 s | 8/8 | | 29.3 KiB | 2.7 KiB | |
| 35 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:31 | 13 ms | 1/1 | 203.2 KiB | | | 2.7 KiB |
| 34 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:31 | 3 ms | 1/1 | 9.3 KiB | | | |
| 33 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:29 | 0.1 s | 8/8 | | 1203.0 B | | |
| 31 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:29 | 3 ms | 1/1 | 9.3 KiB | | | |
| 30 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:28 | 0.1 s | 8/8 | | 29.4 KiB | 2.7 KiB | |
| 29 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:28 | 21 ms | 1/1 | 203.2 KiB | | | 2.7 KiB |
| 28 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:28 | 4 ms | 1/1 | 9.3 KiB | | | |
| 27 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0.. start at NativeMethodAccessorImpl.java:0 +details | 2026/11/08 16:22:26 | 0.1 s | 8/8 | | 1203.0 B | | |
| 25 | id = 49ae262c-df89-4625-847f-3ffba90e0a82 runId = b44a222e-d082-456f-a648-a98c0 | 2026/11/08 16:22:26 | 3 ms | 1/1 | 9.3 KiB | | | |

## Saved Spark SQL query and actual result

### GridPulse - Spark SQL query and result

Recorded execution evidence | outputs/sql/top_days.sql + top_days.csv | Historical telemetry replay, local[4]

```
SELECT to_date(event_time) day, COUNT(*) valid_minutes,
       ROUND(SUM(energy_kwh),3) observed_kwh FROM energy GROUP BY 1
       HAVING COUNT(*) >= 1380 ORDER BY observed_kwh DESC LIMIT 10
```

| day | valid_minutes | observed_kwh |
|---|---|---|
| 2006-12-23 | 1440 | 79.556 |
| 2007-02-03 | 1440 | 67.162 |
| 2006-12-26 | 1440 | 65.569 |
| 2007-02-18 | 1440 | 63.829 |
| 2007-02-04 | 1440 | 59.932 |
| 2007-02-11 | 1440 | 59.520 |
| 2007-03-31 | 1440 | 58.492 |
| 2006-12-31 | 1440 | 58.237 |
| 2007-03-11 | 1440 | 58.011 |
| 2007-01-21 | 1440 | 56.788 |

### GridPulse - Structured Streaming results

Recorded execution evidence | replay_progress.json + replay_hourly.csv | Historical telemetry replay, local[4]

# Recorded micro-batches, hourly aggregates and scoring eligibility

## GridPulse - Structured Streaming results

Recorded execution evidence | replay_progress.json + replay_hourly.csv | Historical telemetry replay, local[4]

### 21,600 real input records | checkpoint restart | batch parity PASS

### Actual micro-batch progress

| batch_id | input_rows | watermark |
|---|---|---|
| 0 | 3600 | 1970-01-01T00:00:00.000Z |
| 2 | 3600 | 2010-01-03T10:00:00.000Z |
| 4 | 3600 | 2010-01-05T22:00:00.000Z |
| 6 | 3600 | 2010-01-08T10:00:00.000Z |
| 8 | 3600 | 2010-01-10T22:00:00.000Z |
| 10 | 3600 | 2010-01-15T14:09:00.000Z |

### Hourly aggregates and scoring eligibility (selected windows)

| start | records | mean_kw | scoring_status | alert | window_state |
|---|---|---|---|---|---|
| 2010-01-12 14:00:00 | 53 | 2.5081 | insufficient_coverage | UNSCORED | finalized |
| 2010-01-18 04:00:00 | 10 | 0.3134 | insufficient_coverage | UNSCORED | provisional |
| 2010-01-01 00:00:00 | 60 | 1.0859 | eligible | False | finalized |
| 2010-01-01 01:00:00 | 60 | 0.6512 | eligible | False | finalized |
| 2010-01-01 02:00:00 | 60 | 0.6346 | eligible | False | finalized |
| 2010-01-01 03:00:00 | 60 | 0.6530 | eligible | False | finalized |